GAZI UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

BM314-SOFTWARE ENGINEERING
Prof. Dr. Hacer Karacan

SPRING 2026

SOFTWARE PROJECT MANAGEMENT PLAN (SPMP)

Project Title: SMarket

Group Members:

Salih Akoğlu-23118080072

Ediz Gün-23118080082

Submission Date: 11.02.2026

Revisions

Version	Date	Description	Author
0.1	**02.02.2026**	Initial project brainstorming and basic feature definition.	Ediz Gün, Salih Akoğlu
0.2	**07.02.2026**	Drafted the Task Breakdown (WBS) and Scheduling	Ediz Gün, Salih Akoğlu
1.0	**09.02.2026**	SPMP Submission	Ediz Gün, Salih Akoğlu

Table of Contents

1. INTRODUCTION 3

1.1 Project Overview ……..3

1.2 Project Deliverables 3

2. PROJECT ORGANIZATION 4

2.1 Software Process Model (Scrum/Incremental) ……4

2.2 Roles and Responsibilities ……5

2.3 Tools and Techniques 5

3. PROJECT MANAGEMENT PLAN 5

3.1 Tasks (Work Breakdown Structure) ……5

3.1.1 Task-1: User Authentication & RBAC …….5

3.1.2 Task-2: Inventory & FIFO Management ……...6

3.1.3 Task-3: Sales & VAT System …….6

3.1.4 Task-4: Automated Supply Chain & Mail API ……7

3.1.5 Task-5: Campaign Engine & Loyalty Points ……..8

3.1.6 Task-6: Profit Calculation Engine …….8

3.1.7 Task-7: System Audit Logs ……..9

3.2 Assignments 9

3.3 Timetable (Sprint Schedule & Gantt Chart) 10

4. ADDITIONAL MATERIAL .....

1.Introduction

1.1 Project Overview

SMarket is an integrated, web-based enterprise resource planning (ERP) solution specifically engineered to digitize and automate the operations of modern retail businesses. The primary objective is to replace inefficient manual tracking methods with a reliable, data-driven system that minimizes financial loss and maximizes operational speed.

The system is designed with a high-performance architecture to address the complexities of inventory and financial management through two specialized portals:

Manager Portal: An administrative command center designed for high-level oversight. It provides comprehensive tools for human resources management (salaries, shift tracking), dynamic campaign definition, and real-time financial analytics, including automated profit/loss reporting.

Cashier Portal: A streamlined, user-friendly interface. It handles real-time stock deductions, automated VAT (KDV) calculations, and integrated customer loyalty point management.

SmartMarket incorporates advanced engineering modules to ensure business continuity and accuracy:

Inventory Intelligence: The system utilizes a strictly enforced FIFO (First-In-First-Out) algorithm. By tracking arrival and expiration dates at the batch level, the engine ensures that older stock is prioritized, significantly reducing waste and ensuring product freshness.

Supply Chain Automation: To prevent stockouts, an Automated Supply Engine monitors inventory levels against critical thresholds. When necessary, the system triggers an integrated Email API to notify suppliers automatically with precise order requirements.

Accountability & Security: A centralized Audit Log records every administrative modification, providing a transparent and immutable history of the system's state to prevent internal fraud and ensure data integrity.

1.2 Project Deliverables

The following documents and products will be delivered according to the project lifecycle:

Software Project Management Plan (SPMP): This document, detailing task allocation, risk management, and the Scrum-based schedule. (Due: March 11, 2026)

Software Requirements Specifications (SRS): A deep dive into functional requirements like FIFO logic, VAT calculations, Role-Based Access Control (RBAC), and the Email API trigger conditions. (Due: April 1, 2026)

Software Design Description (SDD): Detailed ER Diagrams for the database, Sequence Diagrams for the sale process, and high-fidelity UI/UX Mockups for the Manager and Cashier portals. (Due: April 22, 2026)

Software Test Documentation (STD): A report covering unit tests for the Profit Engine and integration tests for the Nodemailer/Mail API to ensure supplier notifications are sent correctly. (Due: May 13, 2026)

Executable Software System: The final source code of the web application, including the Frontend (React), Backend (Node.js), and Database (PostgreSQL) scripts. (Due: May 13, 2026)

Project Presentation: A live demonstration showcasing a full cycle: User Login -> Product Batch Entry -> Automated Discount Trigger -> Final Sale -> Profit Reporting. (Due: May 13, 2026)

2.PROJECT ORGANIZATION

2.1 Software Process Model

The Scrum framework has been selected as the primary development methodology to ensure high adaptability and continuous delivery throughout the lifecycle of the SmartMarket project. Given the complexity of integrating multiple modules—such as the FIFO inventory engine, automated Mail APIs, and financial reporting—an iterative and incremental approach is essential. This methodology allows the development team to build a core functional system first, while providing the flexibility to refine advanced features like loyalty points and audit logs based on real-time testing and evolving system requirements.

Sprint Duration: 2 weeks.

Total Sprints: 5 Sprints

2.2 Roles and Responsibilities

While the system supports internal roles (Manager/Cashier), the development roles are distributed between the two team members to cover all phases of the software lifecycle:

Salih Akoğlu (Backend Developer & QA Engineer): Technically oversees database design, Mail API integration, and core Business Logic. Additionally handles all functional testing of the system components, especially the loyalty points and audit logs.

Ediz Gün (Frontend Developer & Project Manager): Responsible for UI/UX design, Dashboard creation, and Role-based navigation. Responsible for scope definition, risk management, and SPMP updates.

2.3 Tools and Techniques

Development Platform: Web (Cross-platform accessibility).

Frontend Tech: React.js / Next.js (for high-speed Dashboard performance).

Backend Tech: Node.js (Express) or Python (FastAPI).

Database: PostgreSQL (for relational data integrity).

Communication/API: Nodemailer (for Supplier Emailing).

Version Control: Git & GitHub.

3. PROJECT MANAGEMENT PLAN

3.1 Tasks

3.1.1 Task-1: User Authentication & Role-Based Access Control (RBAC)

3.1.1.1 Description: Development of a secure authentication system allowing users to log in as either "Manager" or "Cashier." The system will enforce role-based permissions; Managers can view all staff details (salary, hire date, performance), while Cashiers are restricted to their personal profile and the sales terminal.

3.1.1.2 Deliverables and Milestones: Login interface, encrypted user database, and role-based route protection.

3.1.1.3 Resources Needed: React.js, Node.js (Express), JWT (JSON Web Tokens), Bcrypt library.

3.1.1.4 Dependencies and Constraints: Must be implemented first to secure subsequent modules.

3.1.1.5 Risks and Contingencies: Unauthorized access due to weak password policies. Contingency: Implementing strong password validation and multi-factor simulation.

3.1.2 Task-2: Inventory Management

3.1.2.1 Description: Tracking product entries and exits with specific focus on arrival dates, expiry dates, and batch IDs. Implementing FIFO (First-In-First-Out) logic to prioritize the sale of older stock. Includes functionality for Managers to manually adjust stock levels when necessary.

3.1.2.2 Deliverables and Milestones: Inventory dashboard, FIFO dispatch algorithm, and stock alert system.

3.1.2.3 Resources Needed: PostgreSQL (Batch tracking schema), Backend inventory service.

3.1.2.4 Dependencies and Constraints: Dependent on the finalized database schema.

3.1.2.5 Risks and Contingencies: Logical errors in FIFO batch switching during simultaneous sales. Contingency: Row-level database locking during stock updates.

3.1.3 Task-3: Sales and KDV (VAT) System

3.1.3.1 Description: Development of the Cashier's Point of Sale (POS) interface. The system will support dynamic VAT calculation based on product categories (e.g., 1%, 10%, 20%). Includes a return module that restocks the item and logs the reversal transaction.

3.1.3.2 Deliverables and Milestones: POS UI, VAT calculation engine, and Return processing module.

3.1.3.3 Resources Needed: Frontend components, Financial logic functions.

3.1.3.4 Dependencies and Constraints: Hard dependency on Task-2 (Inventory) for real-time stock availability.

3.1.3.5 Risks and Contingencies: Incorrect VAT rounding. Contingency: Using decimal precision libraries to ensure financial accuracy.

3.1.4 Task-4: Automated Supply Chain and Mail

3.1.4.1 Description: Automating the replenishment process. When stock levels fall below a specific threshold, the system triggers an automatic email to the supplier (Manager's email) containing the product name and required order quantity.

3.1.4.2 Deliverables and Milestones: Automated order trigger service and Mail API integration.

3.1.4.3 Resources Needed: Nodemailer (Mail API), Backend cron-jobs or triggers.

3.1.4.4 Dependencies and Constraints: Requires stable external API connectivity.

3.1.4.5 Risks and Contingencies: Supplier emails being marked as spam or failing to send. Contingency: Implementing a local "Failed Notifications" dashboard for manual oversight.

3.1.5 Task-5: Point System

3.1.5.1 Description: Implementation of a customer loyalty program. Customers earn points based on their purchase total, which are stored in the database as a redeemable digital currency for future transactions.

3.1.5.2 Deliverables and Milestones: Loyalty database, point accrual logic, and redemption interface.

3.1.5.3 Resources Needed: Database point tables, Frontend transaction modal.

3.1.5.4 Dependencies and Constraints: Must be integrated directly into the Sales module (Task-3).

3.1.5.5 Risks and Contingencies: "Points inflation" or logic flaws allowing double redemption. Contingency: Strict transaction-based point validation.

3.1.6 Task-6: Profit Calculation

3.1.6.1 Description: Developing a financial analytics engine for Managers. The module calculates the net profit per item and per period by applying the following logic:

3.1.6.2 Deliverables and Milestones: Profit/Loss analytics dashboard and period-based financial reports.

3.1.6.3 Resources Needed: Data visualization libraries (e.g., Chart.js), Backend aggregation queries.

3.1.6.4 Dependencies and Constraints: Requires accurate input of cost prices from the Inventory module.

3.1.6.5 Risks and Contingencies: Data inconsistency between sales and inventory. Contingency: Daily cross-reference reports for financial auditing.

3.1.7 Task-7: System Logs

3.1.7.1 Description: Creation of an immutable audit log to track all significant administrative actions. The system will record "Who, What, and When" for events such as price changes, stock overrides, or salary updates.

3.1.7.2 Deliverables and Milestones: Audit log table and a searchable Manager-only Log Viewer.

3.1.7.3 Resources Needed: Database logging triggers, Secure log-viewing UI.

3.1.7.4 Dependencies and Constraints: Must log actions from all other modules (Task-1 through Task-6).

3.1.7.5 Risks and Contingencies: Large log volumes affecting database performance. Contingency: Implementing log rotation and indexing for optimized retrieval.

3.2 Assignments

The project responsibilities are distributed among the team members based on their technical expertise and the requirements of the Work Breakdown Structure (WBS). The assignments are as follows:

Salih Akoğlu (Backend Developer & QA Engineer)

Database Architecture: Designing the relational schema for products, batches (FIFO), and user roles.

Business Logic Implementation: Development of core backend engines, including VAT (KDV) calculations, FIFO stock dispatch, and profit reporting.

Integration: Implementation of the Mail API (Nodemailer) for automated supplier notifications.

Quality Assurance (QA): Responsible for drafting the Software Test Documentation (STD). This includes functional testing of the Customer Loyalty Point System and ensuring the integrity of the Audit Logs.

Ediz Gün (Frontend Developer & Project Manager)

UI/UX Design: Designing high-fidelity mockups and the final web interface using modern frontend frameworks.

Dashboard Development: Creating specialized dashboards for both Manager and Cashier roles, including responsive data visualizations.

Project Management: Leading the Scrum process, defining project scope, and performing continuous Risk Management.

Documentation: Responsible for maintaining and updating the SPMP and SRS documents throughout the project lifecycle.

3.3 Timetable

Phase/Sprint	**Time Interval**	**Focus & Key Activities**	**Deliverables & Milestones**
Sprint 1: The Foundation	**Mar 12 – Mar 25**	Environment setup (React/Node.js), Database ERD design, and User Authentication (RBAC) development.	**Basic App Shell (Login & DB Connection)**
Sprint 2: Inventory & FIFO	**Mar 26 – Apr 08**	**Milestone:** SRS Due (Apr 01). Developing Product/Batch management and implementing the FIFO logic for stock dispatch.	**Functional Inventory Module (Batch Tracking)**
Sprint 3: Sales & Finance	**Apr 09 – Apr 22**	**Milestone:** SDD Due (Apr 22). Building the POS interface, VAT engine, and Profit Calculation logic.	**Working POS System (Real-time Reporting)**

Sprint 4: Automation	**Apr 23 – May 06**	Integrating Mail API (Nodemailer), Customer Loyalty Points, Campaign Engine, and System Audit Logs.	**Fully Featured "Smart" System**
Sprint 5: Final Polish	**May 07 – May 13**	**Milestones:** STD & Source Code & Presentation (May 13). Final unit/integration testing and presentation prep.	**Final Executable Software & Documentation**

ADDITIONAL MATERIAL

References

React.js / Next.js: Official documentation for building the high-speed dashboard and specialized web portals. https://react.dev/

Node.js (Express): Official documentation for the backend server environment and secure authentication middleware. https://nodejs.org/

PostgreSQL: Official resource for relational database integrity, row-level locking, and batch tracking schemas. https://www.postgresql.org/docs/

Nodemailer API: Documentation for integrating automated supplier email notifications and cron-job triggers. https://nodemailer.com/

Git & GitHub: Documentation for collaborative version control and Scrum board management. https://docs.github.com/